JavaScript Asinkron: fetch, Promise, dan JSON

Pengembangan Aplikasi Berbasis Web · SIF302 · Semester Gasal 2026/2027

NAMA	NIM	KELAS	TANGGAL
Faiq Adiyatma Prakosa	25523088	B	14/10/2026
WORKSHEET	**P10**	**Pertemuan 10 · JavaScript Asinkron: fetch, Promise, dan JSON**	

Dikerjakan sendiri, bukan berpasangan. Setiap mahasiswa mengerjakan satu halaman profil miliknya sendiri. Anda bebas memilih isi halaman, tetapi harus bisa menjelaskan setiap baris kode yang Anda tulis.

Capaian pembelajaran	Yang dinilai hari ini
Mahasiswa mampu mengambil data dari sebuah API memakai fetch, membaca balasan JSON, memisahkan lapisan data dari lapisan tampilan dalam berkas js/api.js, lalu menampilkan hasilnya di halaman beserta keadaan memuat, berhasil, gagal, dan kosong.	1. Pemanggilan fetch benar dan alamatnya ditetapkan di satu tempat. 2. respons.ok diperiksa sebelum badan balasan dibaca. 3. JSON.parse dan JSON.stringify dipakai pada arah yang tepat. 4. Keempat keadaan antarmuka tampil: memuat, berhasil, gagal, kosong. 5. Galat ditangkap dan pesannya ditampilkan di halaman, tidak sekadar di console. 6. Berkas js/api.js memuat lapisan data dan mengekspor fungsinya, sehingga app.js tidak perlu diubah.

Aturan pengerjaan

Hal	Ketetapan
Bentuk kerja	Individu. Satu mahasiswa, satu halaman, satu topik.
Perkakas	VS Code + peramban Chrome atau Firefox. Panel Console dan Network di DevTools dipakai terus-menerus mulai lembar A.
Berkas awal	Hasil pekerjaan Anda sendiri dari Pertemuan 9, profil.html, seluruh berkas CSS, js/app.js, dan js/dom.js, disalin ke folder worksheet-p10/. Tidak ada berkas contoh, tidak ada kerangka yang disediakan.
Berkas baru	Satu berkas saja: js/api.js. Berkas ini memuat lapisan data dan mengekspornya. app.js tetap seperti semula, hanya menambah satu baris pemanggil.
API	API publik tanpa kunci akses. Bila alamatnya tidak terjangkau dari kelas, pakai satu berkas notisia.json lokal dan sebutkan pilihan itu di README.
Waktu	90 menit di kelas. Bila belum selesai, dilanjutkan di luar kelas paling lambat pukul 23.59 pada hari yang sama.
Penyerahan	Seluruh isi worksheet-p10/ diunggah ke repositori GitHub Anda sendiri, di dalam folder semester ~/pabw/. Periksa halaman repositori Anda, tidak cukup folder di laptop.

Anggaran waktu 90 menit

Menit	Lembar	Yang Anda kerjakan	Kalau waktu habis, berhenti di sini
0-18	A	Salin hasil Pertemuan 9 ke worksheet-p10/, buat js/api.js, panggilan fetch pertama, periksa respons.ok	Objek Response tampil di Console
18-36	B	Ubah rantai then menjadi async/await, satu try/catch, buktikan galatnya tertangkap	Galat jaringan tertangkap dan tampil di halaman
36-54	C	Baca balasan dengan json(), petakan bentuk datanya, kenali arah stringify	Kunci data teridentifikasi dan terdokumentasi
54-74	D	Telusuri larik, gambar kartu ke halaman, keempat keadaan antarmuka	Daftar tampil di halaman dari data
74-96	E	Tiga kasus sulit: status 404, await di luar fungsi async, badan bukan JSON	Tiga kasus ditangani + catatan
96-150	F	Periksa satu per satu, satu baris untuk diingat, tiket keluar, penilaian mandiri, README, catat bagian P13	Lembar F terisi dan tersimpan

Selalu kerjakan berurutan. Setiap lembar mengandaikan lembar sebelumnya sudah selesai, dan setiap keadaan antarmuka dibuktikan dengan tangkapan layar. Kolom paling kanan menunjukkan laju, bukan tenggat.

WORKSHEET **A** **Panggilan pertama dan status balasan**

fetch TIDAK MELEMPAR GALAT PADA STATUS 404, PERIKSA respons.ok

Panggilan pertama sering dianggap berhasil karena tidak ada galat di Console. Padahal balasan bisa berstatus 404 atau 500 dan tetap dianggap berhasil oleh fetch. Satu pemeriksaan singkat menutup seluruh celah itu.

A.1 Berkas yang harus ada sebelum Anda mulai

Salin seluruh hasil Pertemuan 9 ke folder worksheet-p10/. Periksa satu per satu, pekerjaan hari ini bertumpu pada berkas-berkas itu, dan yang hilang akan terasa sebagai galat 404 di Console.

Berkas	Dari	Ada?	Jumlah baris
profil.html	P9	ada	
Seluruh berkas CSS (enam berkas)	P4-P6	ada	
js/app.js	P8	ada	
js/dom.js	P9	ada	

A.2 Tetapkan alamat di satu tempat

Alamat API ditulis satu kali saja, di bagian atas berkas. Bila alamatnya berubah nanti, hanya satu baris yang perlu disunting, bukan setiap tempat yang memanggilnya.

```
// js/api.js, bagian atas berkas
const ALAMAT = 'https://api.contoh.tl/notisia';
// ganti dengan alamat API publik yang Anda pilih
```

Hal yang diperiksa	Tanda benar
Skema alamat	memakai https, bukan http
Satu tempat	alamat tidak ditulis ulang di fungsi lain
Tanpa kunci akses	tidak ada apiKey atau token di dalam kode
Nama berkas	alamat berada di js/api.js, bukan di js/app.js

A.3 Panggil, periksa, lalu buktikan di panel Network

Panggil fetch, lalu periksa respons.ok sebelum membaca badannya. Isi tabel berikut dari panggilan Anda sendiri.

```
async function muat() {
  const respons = await fetch(ALAMAT);
  console.log('status', respons.status, 'ok', respons.ok);
  if (!respons.ok) {
    throw new Error('status ' + respons.status);
  }
  return respons.json();
}
```

Yang dicatat	Nilai yang saya dapat
Alamat API yang saya pakai	https://jsonplaceholder.typicode.com/posts
Status balasan (200, 404, dan seterusnya)	200, jika hasil pengujianmu menunjukkan 200
Nilai respons.ok	true, jika statusnya 200
Jenis isi (Content-Type)	Umumnya application/json; charset=utf-8; periksa tab Network → Headers untuk memastikan

Buka satu panggilan di panel Network. Periksa tab Headers untuk status dan jenis isi, tab Response untuk badan mentahnya. Badan mentah inilah yang nanti dibaca json(), jadi kenali bentuknya sebelum menulis kode penggambar.

SIMPAN KE GIT - SETELAH LEMBAR A

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: panggilan pertama dan status balasan
```

Lembar A sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

A - simpan perubahan

Cara memastikan: buka Console, panggil fungsi muat() Anda. Status dan nilai respons.ok tampil, dan tidak ada tulisan merah.

WORKSHEET **B** **async, await, dan penanganan galat**

SATU try/catch MENUTUP SELURUH RANTAI

Rantai then yang panjang sulit dibaca dan galatnya mudah terlewat. async dan await menuliskan alur yang sama secara mendatar, dan satu blok try/catch menangkap galat dari seluruh langkah di dalamnya.

B.1 Ubah rantai menjadi async/await

Tulis fungsi async, lalu bungkus seluruh langkahnya dengan try/catch. Bandingkan bentuknya dengan rantai then pada slide 9 dan 14.

```
export async function ambilNotisia() {
  try {
    const respons = await fetch(ALAMAT);
    if (!respons.ok) throw new Error('status ' + respons.status);
    const data = await respons.json();
    return data;
  } catch (galat) {
    console.error('gagal memuat', galat.message);
    throw galat;
  }
}
```

await hanya boleh dipakai di dalam fungsi async. Pemanggil di luar fungsi itu masih perlu menangani penolakannya, di pekerjaan Anda, pemanggilnya adalah app.js, dan di sanalah blok try/catch kedua dipasang.

B.2 Kenali tiga kegagalan

Jenis kegagalan	Yang terjadi pada Promise	Cara menanganinya
Jaringan mati atau alamat salah	Promise ditolak	catch menangkap galat, pesannya ditampilkan
Status 404 atau 500	Promise tetap berhasil	periksa respons.ok, lalu lempar galat sendiri
Badan bukan JSON	json() menolak	catch menangkap galat, pesannya ditampilkan
Balasan sah tetapi tanpa butir	Promise berhasil, larik kosong	tampilkan keadaan kosong, jangan layar putih

B.3 Buktikan galatnya tertangkap

Yang saya uji	Hasil yang benar	Hasil yang saya dapat
Alamat diubah menjadi alamat yang tidak ada	Pesan galat tampil di halaman, Console bersih dari galat tak tertangani	Belum dikonfirmasi
Jaringan dimatikan di panel Network	Pesan galat tampil, bukan layar kosong	Belum dikonfirmasi
Alamat dikembalikan seperti semula	Daftar tampil kembali tanpa menyunting kode lain	Perlu diperiksa kembali

Sengaja rusakkan, buktikan, lalu kembalikan. Uji yang tidak dikembalikan akan tampak sebagai kerusakan pada pertemuan berikutnya, dan Anda yang akan menelusurinya kembali.

SIMPAN KE GIT - SETELAH LEMBAR B

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: async, await, dan satu try/catch
```

Lembar B sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

B - simpan perubahan

Cara memastikan: ubah alamat menjadi alamat yang salah, muat ulang halaman. Pesan galat tampil di halaman, dan Console tidak menampilkan galat tak tertangani.

WORKSHEET **C** **Membaca JSON**

TEKS MASUK, NILAI KELUAR, DAN SEBALIKNYA

Balasan jaringan datang sebagai teks. json() mengubahnya menjadi nilai JavaScript sebelum dapat dipakai. Arah sebaliknya dipakai saat mengirim data ke peladen, dan di situlah JSON.stringify bekerja.

C.1 Dua arah

Baca badan balasan dengan json(), dan kirim data dengan stringify. Perhatikan jenis nilai yang keluar pada tiap arah, salah arah adalah penyebab undefined yang paling sering.

```
// teks menjadi objek
const data = await respons.json();
console.log(data.titulu);

// objek menjadi teks
const badan = JSON.stringify({ titulu: 'Avisu', nivel: 2 });
console.log(typeof badan);   // string
```

Yang dipakai	Arah	Hasilnya	Kapan dipakai
respons.json()	teks → nilai	bentuk data yang bisa diakses dengan kunci	membaca balasan peladen
JSON.parse(teks)	teks → nilai	sama, tetapi untuk teks yang Anda pegang sendiri	membaca localStorage atau berkas JSON lokal
JSON.stringify(nilai)	nilai → teks	teks yang siap dikirim atau disimpan	mengirim data, menyimpan ke localStorage

C.2 Peta data saya

Tulis bentuk balasan API Anda sendiri. Kolom kiri adalah nama kunci, kolom tengah jenis nilainya dan contoh isinya, kolom kanan cara mengaksesnya di dalam kode.

Nama kunci	Jenis nilai dan contoh	Cara mengaksesnya
User id	objek	data.____
Id	larik	data.____.forEach(____)
Title	teks atau angka	data.____
body	benar atau salah, atau null	data.____

C.3 Periksa bentuknya sebelum menggambar

Panggil console.log(data) sekali sebelum menulis kode penggambar. Membaca bentuk data lebih dulu menghemat waktu, karena kunci yang salah hanya menghasilkan undefined tanpa galat yang jelas.

SIMPAN KE GIT - SETELAH LEMBAR C

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: membaca JSON dua arah
```

Lembar C sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

C - simpan perubahan

Cara memastikan: tulis nama kunci Anda di Console satu per satu. Tidak ada satu pun yang bernilai undefined.

WORKSHEET **D** **Menampilkan data ke halaman**

PISAHKAN PENGAMBILAN DATA DARI PENGGAMBARAN

Fungsi pengambil data hanya bertugas mengambil dan memeriksa. Fungsi penggambar hanya bertugas menyusun elemen. Dipisah begitu, halaman mudah diuji ketika alamat peladennya berubah, dan itulah bentuk yang dipakai lagi di Pertemuan 11.

D.1 Telusuri larik dan gambar kartu

Telusuri larik hasil parse, buat satu elemen untuk tiap butir, lalu pasang ke wadah yang sudah ada di HTML. Gunakan textContent, bukan innerHTML, untuk isi yang datang dari jaringan.

```
import { ambilNotisia } from './api.js';

const wadah = document.querySelector('#daftar');

function gambar(daftar) {
  wadah.textContent = '';
  daftar.forEach(function (n) {
    const kartu = document.createElement('article');
    const judul = document.createElement('h3');
    judul.textContent = n.titulu;
    kartu.appendChild(judul);
    wadah.appendChild(kartu);
  });
}

async function tampilkan() {
  wadah.textContent = 'Memuat...';
  try {
    const daftar = await ambilNotisia();
    if (daftar.length === 0) {
      wadah.textContent = 'Belum ada notisia';
      return;
    }
    gambar(daftar);
  } catch (galat) {
    wadah.textContent = 'Gagal memuat data: ' + galat.message;
  }
}

tampilkan();
```

D.2 Keempat keadaan antarmuka

Isi tabel berikut dengan cara Anda menampilkan tiap keadaan. Keadaan kosong sering terlupa, padahal balasan yang sah bisa berisi larik kosong.

Keadaan	Kapan terjadi	Apa yang tampil di halaman
Memuat	sebelum balasan datang	Memuat data…
Berhasil	balasan berisi data	Kartu berisi judul dan isi postingan
Gagal	jaringan gagal atau status bukan 2xx	Pesan Gagal memuat data: …
Kosong	balasan sah tetapi tanpa butir	Belum ada data.

D.3 Periksa batas lapisan

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
js/api.js memanggil fetch dan mengekspor fungsinya	satu export terlihat di bagian atas berkas	Periksa kembali berkas
js/app.js tidak memanggil fetch sendiri	kata fetch hanya ada di js/api.js	belum sesuai jika fetch masih ada di app.js; periksa

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
app.js menangani penolakan dari api.js	blok try/catch kedua ada di pemanggil	Sudah ada pada kode sebelumnya
Wadah dikosongkan sebelum digambar ulang	jumlah kartu tidak berlipat saat dimuat ulang	sudah diterapkan pada fungsi penggambar
Nama fungsi yang diekspor tercatat	nama itu yang akan ditulis di berkas pengimpor Pertemuan 11	ya

SIMPAN KE GIT - SETELAH LEMBAR D

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: data dari jaringan tampil di halaman
```

Lembar D sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

D - simpan perubahan

Cara memastikan: muat ulang halaman tanpa menyentuh Console. Kartu tampil dari data, dan tidak ada kartu yang berlipat.

WORKSHEET **E** **Tiga kasus sulit**

TIDAK ADA GALAT DI CONSOLE, TETAPI HALAMAN DIAM

Ketiga kasus berikut tidak selalu menimbulkan galat. Yang terlihat hanya halaman yang diam atau kosong, sehingga gejalanya perlu dibaca lebih dulu sebelum kodenya disentuh.

E.1 Status 404 tidak melempar galat

fetch hanya menolak Promise ketika permintaannya gagal, jaringan mati, alamat tidak dapat dijangkau, atau permintaan diblokir. Status 404 dan 500 tetap dianggap berhasil. Satu pemeriksaan respons.ok menutup celah itu sebelum badannya dibaca.

Gejala	Sebab yang paling sering	Yang dilakukan
Halaman menampilkan kartu kosong	badan 404 dibaca sebagai data	periksa respons.ok sebelum memanggil json()
Daftar berisi undefined	kunci data salah, tetapi statusnya 200	cetak console.log(data) sekali untuk melihat bentuknya
json() melempar SyntaxError	badan bukan JSON, sering halaman galat HTML	periksa Content-Type di panel Network

E.2 await di luar fungsi async

await hanya sah di dalam fungsi async. Bila dipasang di luar, berkasnya gagal diurai dan seluruh skrip ikut berhenti, tidak cukup satu baris. Gejalanya mudah dikenali: tidak satu pun kode di berkas itu berjalan.

```
// salah: berkas gagal diurai
const data = await fetch(ALAMAT);

// benar: dibungkus fungsi async
async function muat() {
  const data = await fetch(ALAMAT);
}
```

E.3 Balasan sah tetapi tanpa butir

Balasan dengan larik kosong tidak menimbulkan galat apa pun. Halaman hanya tampak kosong, dan pengguna tidak tahu apakah datanya belum ada atau kodenya salah. Keadaan kosong perlu ditulis sebagai keadaan tersendiri.

Tulis pesan kosong yang menjelaskan keadaan yang sedang terjadi. "Belum ada notisia" lebih berguna daripada halaman putih, dan itu yang membedakan halaman yang selesai dari halaman yang setengah jadi.

E.4 Catat kasus yang saya temui

Gejala yang saya lihat	Sebabnya	Cara saya memperbaiki
dom.js tidak menemukan ekspor daftarProyek	Nama atau lokasi ekspor tidak sesuai	Memisahkan data ke data.js dan memperbaiki baris import
Data kosong tidak menampilkan kartu	Larik tidak memiliki butir	Menampilkan pesan Belum ada data.
Status 404	Alamat API tidak ditemukan	Memeriksa respons.ok dan menampilkan pesan galat

SIMPAN KE GIT - SETELAH LEMBAR E

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: tiga kasus sulit ditangani
```

Lembar E sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

E - simpan perubahan

Cara memastikan: rusakkan alamat, matikan jaringan, dan kosongkan data. Ketiga keadaan tampil sebagai pesan di halaman, bukan layar putih.

WORKSHEET **F** **Periksa, tiket keluar, penilaian mandiri**

SETIAP KEADAAN DIBUKTIKAN DENGAN TANGKAPAN LAYAR

Pemeriksaan dilakukan pada keempat keadaan, dan setiap keadaan disimpan sebagai tangkapan layar. Tanpa bukti itu, halaman yang hanya bekerja sekali sulit ditelusuri kembali.

F.1 Periksa satu per satu

Periksa	Cara memeriksa	Lolos
Alamat di satu tempat	lihat bagian atas js/api.js	☐
respons.ok diperiksa	lihat fungsi pengambil di js/api.js	☐
Galat ditangkap	matikan jaringan, muat ulang halaman	☐
Keadaan memuat tampil	perlambat jaringan di panel Network	☐
Keadaan kosong tampil	panggil gambar([]) dari Console	x
Data tampil sebagai elemen	periksa wadah di panel Elements	☐
Batas lapisan terjaga	kata fetch hanya ada di js/api.js	☐
Tidak ada galat di Console	baca panel Console	☐
Pengungkapan AI	README menyebutkan bagian mana yang dibantu AI dan bagian mana yang saya kerjakan sendiri	☐
Keaslian	topik dan isi halaman ini milik saya sendiri, tidak menyalin pekerjaan mahasiswa lain	☐

F.2 Bukti empat keadaan

Keadaan	Tangkapan layar disimpan	Catatan
Berhasil		
Gagal		
Kosong atau memuat		

F.3 Satu baris untuk diingat

Tuliskan satu potongan kode yang paling sering Anda pakai hari ini, dan tuliskan pada bagian mana potongan itu dipakai. Satu baris sudah cukup.

Potongan kode:	Dipakai pada:

F.4 Penilaian mandiri

Bagian	Bobot	Nilai saya	Bukti
Alur data dan pemeriksaan status	25	25	
Pembacaan JSON dan pemisahan lapisan	25	25	
Keadaan antarmuka	20	20	
Penanganan galat	20	20	
Kebersihan kode, deklarasi AI, dan bukti	10	10	
TOTAL	100	100	

F.5 Tiket keluar

Jawab tiga pertanyaan berikut, satu baris masing-masing. Tiket ini dibaca pengampu sebagai catatan pemahaman Anda.

Pertanyaan	Jawaban
Mengapa fetch tidak melempar galat pada status 404	Karena 404 adalah respons HTTP yang diterima, sehingga fetch() tidak otomatis menolak Promise. Kita harus memeriksa respons.ok dan melempar galat sendiri.

Pertanyaan	Jawaban
Di mana await boleh dipakai, dan di mana tidak	await boleh digunakan di dalam fungsi async atau di tingkat teratas modul yang mendukung top-level await; tidak boleh digunakan sembarangan di dalam fungsi biasa.
Satu keadaan antarmuka yang paling mudah terlupa dan cara Anda menutupnya	Keadaan kosong sering terlupa. Saya menanganinya dengan memeriksa daftar.length === 0 dan menampilkan pesan Belum ada data..

F.6 Catatan untuk pengampu

Bagian yang paling sulit:

Bagian yang paling sulit bagi saya adalah menghubungkan modul JavaScript dengan API tanpa mengganggu fitur yang sudah ada. Saya juga perlu memastikan bahwa error ditangani dan data ditampilkan dengan benar.

Yang ingin saya bahas di kelas berikutnya:

Saya ingin mempelajari lebih lanjut cara menangani berbagai jenis error API, menguji keadaan antarmuka, dan memisahkan lapisan data dengan lapisan tampilan.

Tanda tangan: _____ *Tanggal:* _____ *Jam selesai:* _____

SIMPAN KE GIT - SETELAH LEMBAR F

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: periksa, lalu simpan
```

Lembar F sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

F - simpan perubahan

Cara memastikan: buka README di repositori Anda. Bagian Pertemuan 10 sudah memuat deklarasi AI, dan commit terakhir adalah commit terbaru.

WORKSHEET **H** **Lampiran A, rubrik penilaian**

RUBRIK PENILAIAN total 100

Aspek	Bobot	Nilai 100, sangat baik	Nilai 65, memadai	Nilai 35, kurang
Alur data dan pemeriksaan status	25%	Alamat di satu tempat; respons.ok diperiksa sebelum badan dibaca; fungsi pengambil berada di js/api.js dan diekspor	Panggilan berjalan, tetapi respons.ok atau pemisahan berkas belum lengkap	fetch belum benar, atau status balasan tidak pernah diperiksa
Pembacaan JSON dan pemisahan lapisan	25%	json() dan stringify dipakai pada arah yang tepat; app.js memanggil api.js tanpa memanggil fetch sendiri	Pembacaan benar, tetapi fetch masih ada di app.js atau stringify belum dipakai	Bentuk data ditebak tanpa diperiksa, sehingga banyak nilai undefined
Keadaan antarmuka	20%	Keempat keadaan tampil: memuat, berhasil, gagal, kosong	Tiga keadaan tampil, keadaan kosong belum ada	Hanya keadaan berhasil yang ditangani
Penanganan galat	20%	Satu try/catch menutup seluruh rantai; pesan galat tampil di halaman, tidak sekadar di console	Galat tertangkap, tetapi pesannya hanya di console	Galat tak tertangani, halaman diam tanpa penjelasan
Kebersihan kode, deklarasi AI, dan bukti	10%	Penamaan konsisten, batas api.js dan app.js jelas; deklarasi AI terisi di README; tangkapan layar empat keadaan disertakan	Kode rapi tetapi deklarasi AI atau bukti belum lengkap	Deklarasi AI kosong, atau pekerjaan tidak tersimpan di repositori

Yang dinilai adalah pekerjaan Anda sendiri. Topik yang berbeda antar mahasiswa membuat penilaian tidak bisa dibandingkan antar pekerjaan, karena itu yang diperiksa adalah apakah setiap bagian bekerja sebagaimana dimaksud, bukan apakah ia sama dengan contoh.

WORKSHEET H — Lampiran B, rujukan cepat dan kalau macet

REFERENSI CEPAT ASINKRON — hafalkan yang ini

Yang dipakai	Perannya	Kapan dipakai
fetch(url)	Mengirim permintaan HTTP; hasilnya Promise berisi Response	Ketika mengambil data dari API
respons.ok	benar bila status 2xx, tidak ada galat otomatis pada 404	Ketika memeriksa keberhasilan respons
respons.status	Kode status balasan: 200, 404, 500, dan seterusnya	Ketika mengetahui status seperti 200, 404, atau 500
respons.json()	Membaca badan balasan sebagai JSON; hasilnya Promise	Ketika mengubah respons API menjadi data JavaScript
await	Menunggu Promise selesai; hanya di dalam fungsi async	Di dalam fungsi async atau modul yang mendukung top-level await
async function	Menandai fungsi yang boleh memuat await	Ketika fungsi menggunakan await
try / catch	Menangkap penolakan Promise di sekitarnya	Ketika operasi asinkron dapat gagal
throw new Error()	Membuat galat sendiri, misalnya untuk status bukan 2xx	Ketika status respons bukan 2xx
Promise.all([...])	Menunggu beberapa Promise sekaligus	Ketika beberapa permintaan independen perlu dijalankan bersama
JSON.parse(teks)	Teks menjadi nilai JavaScript	Ketika membaca teks JSON yang dimiliki sendiri
JSON.stringify(nilai)	Nilai menjadi teks JSON	Ketika mengirim atau menyimpan data JSON
export / import	Menyerahkan fungsi ke berkas lain, dipakai js/api.js	Ketika menggunakan fungsi dari berkas JavaScript lain

KALAU MACET — cari gejalanya di sini

Gejala	Penyebab yang paling sering	Yang dilakukan
Console: 404 untuk api.js	Jalur di atribut src tidak cocok dengan letak berkasnya	Pastikan api.js berada di dalam folder js/, huruf besar-kecilnya sama
Console: Cannot read properties of undefined	Kunci data salah, tetapi status balasan 200	Cetak console.log(data) sekali, lalu tulis kunci yang benar-benar ada
Kartu kosong padahal Console bersih	Badan 404 dibaca sebagai data karena respons.ok tidak diperiksa	Periksa respons.ok sebelum memanggil json()
json() melempar SyntaxError	Badan bukan JSON, sering halaman galat HTML	Periksa Content-Type di panel Network
Seluruh berkas berhenti, Console tanpa pesan jelas	await ditulis di luar fungsi async	Bungkus pemanggilnya dengan async function
Halaman diam tanpa penjelasan	Penolakan Promise tidak ditangkap pemanggil	Tambahkan try/catch di app.js dan tampilkan pesannya di halaman
Daftar kosong tanpa keterangan	Balasan sah berisi larik kosong	Tampilkan keadaan kosong ketika panjang lariknya nol
Kartu berlipat tiap kali dimuat ulang	Wadah tidak dikosongkan sebelum digambar	Tambahkan wadah.textContent = '' di baris pertama penggambar
Isi kartu tampil sebagai tag, bukan teks	Isi dimasukkan dengan innerHTML	Ganti dengan textContent
CORS: permintaan diblokir	API tidak mengizinkan asal halaman Anda	Pakai API lain yang mengizinkan CORS, atau berkas notisia.json lokal
Fetch gagal hanya di berkas file://	Halaman dibuka dengan klik dua kali	Jalankan lewat server lokal: python3 -m http.server 8000

Halaman yang Anda bangun hari ini adalah bahan ujian CPMK Blok 2 pada Pertemuan 13 nanti: pemilihan elemen, render dari data, event delegation, dan pengambilan data dari jaringan semuanya diuji di sana. Di Pertemuan 11, js/api.js yang Anda buat hari ini diimpor apa adanya, jadi pilih nama fungsi ekspornya dengan sadar: nama itulah yang ditulis di berkas pengimpor. Simpan pekerjaan ini rapi di repositori Anda.

LAMPIRAN: MENYIMPAN PEKERJAAN ANDA KE GIT — sekali saja, bukan tiap pertemuan

Lampiran ini bukan pengantar Git. Panduan Git dan GitHub yang sudah Anda pakai sejak Pertemuan 3 masih berlaku seluruhnya; bagian ini hanya mengulang tiga perintah yang paling sering dipakai, karena mulai hari ini ada berkas baru yang harus ikut disimpan.

Tiga perintah yang akan Anda pakai terus

Perintah	Artinya	Kapan dijalankan
git add .	Menandai seluruh perubahan di folder ini untuk disimpan	Setiap kali selesai satu lembar
git commit -m "pesan"	Menyimpan perubahan itu sebagai satu riwayat	Langsung setelah git add
git push	Mengirim riwayat itu ke GitHub	Setelah commit, supaya dosen bisa melihatnya

Titik pada git add . Titik itu berarti "seluruh berkas di folder ini". Tanpa titik, Git tidak tahu apa yang harus disimpan. Hal yang sama berlaku untuk berkas api.js yang baru Anda buat, ia ikut tersimpan karena berada di dalam folder semester.

Berapa kali harus menyimpan?

Tidak ada angka yang benar, tetapi patokannya begini: bila Anda sudah menyelesaikan satu hal yang berarti dan bisa Anda jelaskan, simpanlah. Pada pertemuan ini minimal dua kali: setelah data tampil di halaman, dan setelah keempat keadaan antarmuka selesai.

Kalau ini terjadi	Yang Anda lakukan
Sudah menulis banyak, belum pernah commit	Simpan sekarang juga
Salah commit, belum push	Perbaiki lalu commit lagi, jangan hapus repositori
Sudah push, lalu menemukan kekeliruan	Perbaiki berkasnya dan commit lagi; riwayat lama tidak hilang dan itu wajar

Yang dinilai bukan jumlah commit. Menyimpan sering membuat pekerjaan Anda tidak bergantung pada satu berkas di satu komputer, dan pada pertemuan ujian, repositori itulah yang menentukan apakah pekerjaan Anda bisa dinilai.